TVWS-DB User Manual.

Version 1.0.0

Edilson Filho, Thiago Nóbrega e Carlos Silva

Correspondent: edilsonfilho@lesc.ufc.br

Contents

1 Introduction

The present report presents the progress made in the development of the API that functions as a communication bridge through which messages travel between the TVWS devices and the Database. It is organized as follows:

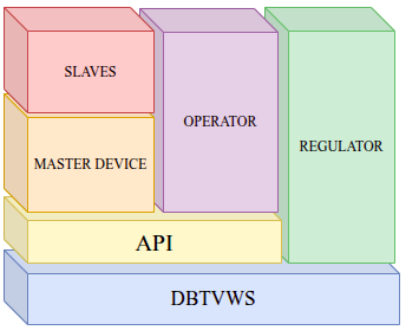
- API Modules , Apresents an overview of the services provided by the API, as well as its operation and configuration modules.
- Messages from PAWS which describes how PAWS and RFC 7545[1] characterize the main messages exchanged between the TVWS device and the Database.
- Customizations section discusses specific adjustments implemented to tailor the solution.
- Deploy is a pratical guide for instantiating the API and integrating in with the database service
- Tests are a portfolio of scripts that automate the diagnosis of the API deployment.
- Conclusion.

2 API Modules

This section presents the API in a modular manner, as well as its relationship with other parts of the system. Each subsection sometimes provides an overview and, at other times, a detailed description of its contents. References to the code repositories are provided to allow the reader to consult the source code when necessary.

2.1 All parts

The Figure 1 illustrates which part communicate with each of the other module of the solution.



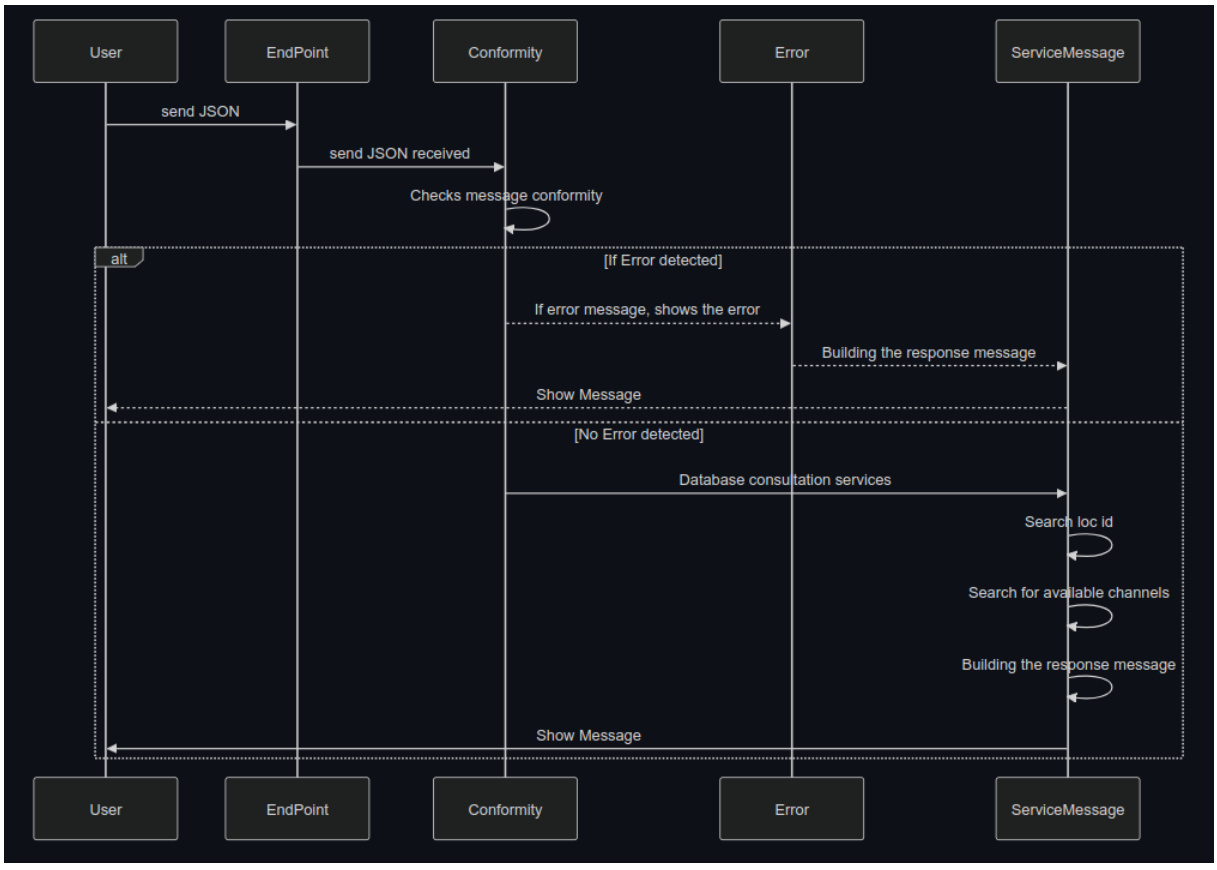
Figures 1: All partes of the TVWS Solution

[1] https://www.rfc-editor.org/rfc/rfc7545.html

The figure highlights the interfaces with which the API module interacts. With the *Regulator*, as it both controls the database and customizes and provides the API service. The *Operator*, which is regulated by the *Regulator*, manages the *Masters* and provides services to the *Slaves* when accessing the API. The *Master* Device provides services to the *Slave* and is managed by the *Operator* in order to access the API with authorization.

2.2 Sub-modules

The communication between the master and the database goes through an important layer in our implementation, compliant with RFC 7545. This layer, referred to here simply as the API, handles the process of verifying messages and their *compliance* with PAWS, checks if the *location* is within the defined coverage zone, queries the *channels*, and performs the necessary calculations for channel searching. A high-level representation is presented Figure 2.

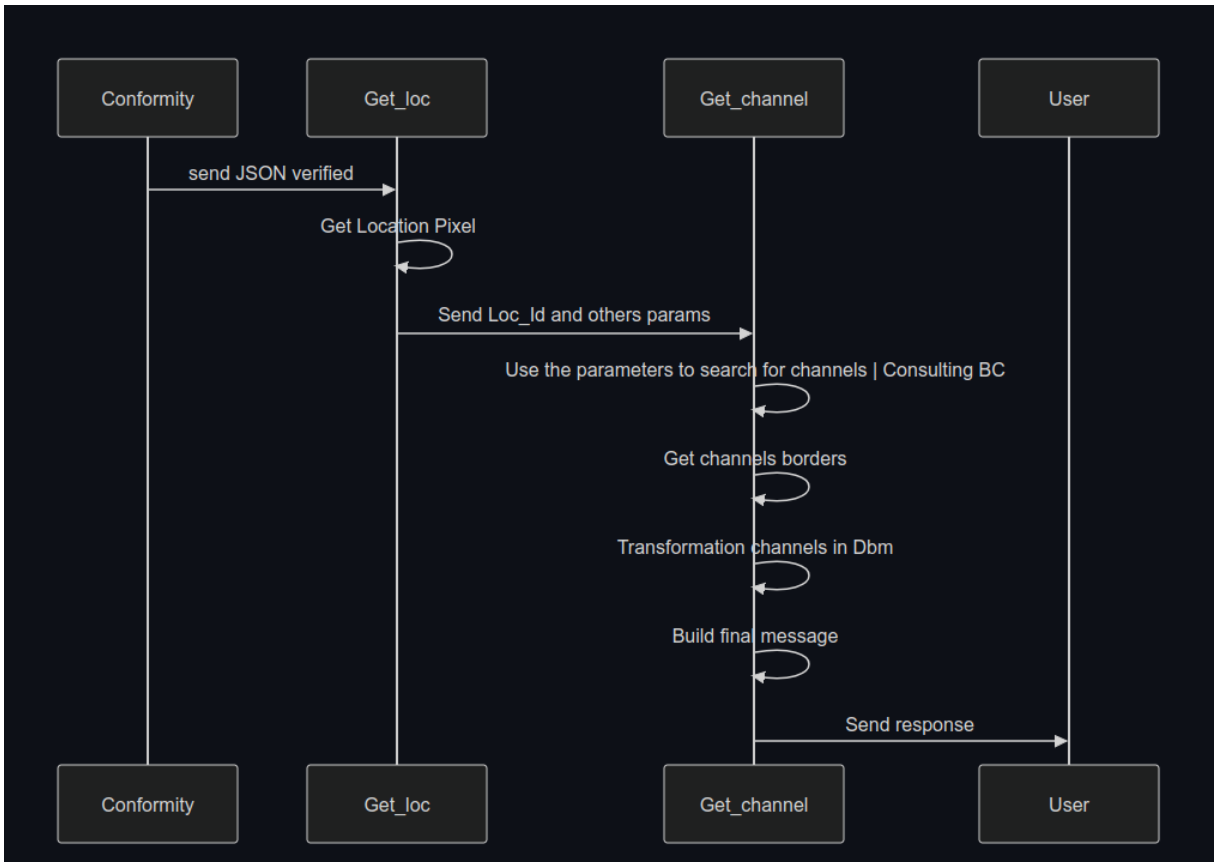


Figures 2: API Modules

User can be interpreted as a *device*. The *EndPoint* is essentially the URL used to provide the service. The *Conformity*, *Error*, and *ServiceMessage* modules are components of the API. Note that after verifying the message's conformity with RFC 7545, two scenarios are possible. The first is the error scenario, where the API will identify the error and send the device information about what went wrong, including potential solutions indicated in the message itself (such as missing or incorrect parameters). The second scenario is a successful conformity verification, where the *ServiceMessage* module will carry out the following steps: i) check the pixel on the

map and return the requester's *loc_id* (which will be used to verify service coverage—if no loc_id is found for the requester, an error message will notify that no coverage is available for that location); and ii), after identification or *loc_id*, is the *search for available channels* and the necessary calculations for coverage, interference protection, and other results. The final step of this module is *Building the response message* (in this case, *AVAIL_SPEC_RESP*) to send to *User*.

The Figure 3 presents, in more detail, the interaction of the modules and the calculations steps. The *ServiceMessage* is divided into a) *Get_loc* and b) *Get_channel*. As mentioned earlier, *Get_loc* searches the map for the requester's unique location identifier. If successful, this value is sent to the service that will perform the *Blockchain* (BC) query and calculations such as *Get_channels borders*, *Transformation channels in dBm*, and *Build final message*.



Figures 3: ServiceMessage sub-modules

The API is multi-platform: In addition to providing the services required to respond to PAWS messages, a range of other functionalities is offered through a set of routes in an environment referred to as administrative.

2.3 API Services

The API was developed to provide support and services for both PAWS messages and solution management and integration services, referred to as administrative services. Accordingly,

in addition to the main URL, the path segments */paws* and */admin* direct the API to handle PAWS messages and management requests, respectively.

3 PAWS Messages

To obtain the available spectrum from the geolocation database, a master device sends a request within the *available spectrum query procedure* that contains its location and any parameters required by the ruleset (e.g., device identifier, capabilities, and characteristics). The geolocation database returns a response that describes which frequencies are available, at what permissible operating power levels, and a schedule of when they are available. In the PAWS protocol (defined in RFC 7545), messages are encoded in JSON, and the *id* field is a mandatory field for all requests and responses following version *2.0* of the protocol.

Note

The API focuses on responding to the requester. That is, in the case of a message where the locations of the slave and master are sent, the API will consider the location of the antenna to respond, including the antenna information.

3.1 Function of the id Field

The *id* field in PAWS messages serves as a *unique identifier for the request or response*[2]. Its primary function is to allow the client (White-Space device) and the server (White-Space database) to correctly associate a response with a previous request. Essentially, it ensures that both parties can unequivocally track interactions between the client and the database, especially in scenarios where multiple messages are being exchanged simultaneously. Structure and Usage:

- **Request**: When the device sends a request to the database, it includes an ïd"field containing a unique value for that specific request.
- **Response**: The database, when responding to the request, includes the same ïd"value in its response, allowing the client device to know which request the response refers to.

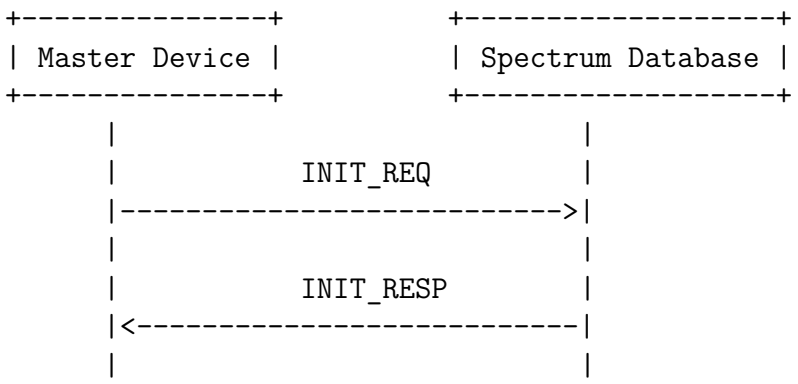
3.2 Importance of the id Field

The *id* field is essential in distributed systems and *JSON-RPC*[3] message exchanges, like in PAWS, to ensure that communication is accurate and that the client can easily track its requests and corresponding responses. This is particularly important in scenarios with multiple requests, allowing the device to maintain control over each interaction with the spectrum database.

[2]See Sec 6.1 in https://www.rfc-editor.org/rfc/rfc7545.html#page-13
[3]JSON-RPC is a lightweight and simple remote procedure call (RPC) protocol based on the JSON.

3.3 Initialization

The RFC recomends that the initialization procedure informs the Master Device of specific ruleset supported by the database, such as threshold distances, antenna parameters, time periods beyond which the device must update its available-spectrum data. Figure 4 present the flow of messages.



Figures 4: Sequence of messages exchanged between Master Device and Spectrum Database.

This message has two parts:

- **INIT_REQ** : initialization request message sent by the Master Device
- **INIT_RESP** : initialization response message sent by the Spectrum Database

In the following subsections, we present our implementation of the INIT Request and Response messages as defined in RFC 7545[4]. Subsection **??** delves into the mechanisms established to maintain a history of all messages exchanged between peer devices, stateless messages named sections. These sections are delineated by specific events or message types, such as the initiation, that will be re-used in the subsequent messages.

3.3.1. INIT_REQ

The *INIT_REQ* is an initialization request message sent by a White Space Device (WSD) to the Spectrum Database Service (SBS), as defined in RFC 7545. The purpose of this request is to allow the device to obtain information about the availability of TV White Space (TVWS) spectrum in its geographic area and initiate the process of operating in that spectrum. The *INIT_REQ* is the first message exchanged in the protocol to establish a connection between the WSD and the database. The *INIT_REQ* follows the following format:

```
{
    "jsonrpc": "2.0",
    "method": "spectrum.paws.init",
    "params": {
     "type": "INIT_REQ",
     "version": "1.0",
```

[4] https://www.rfc-editor.org/rfc/rfc7545.html

```
    "deviceDesc": {
     "serialNumber": "4fbf-bae3-c8c5706745e3",
     "fccId": "94d8610a-9eea",
     "rulesetIds": ["FccTvBandWhiteSpace-2010"]
    },
    "location": {
     "point": {
      "center": {"latitude": -3.870961, "longitude": -38.664911}
     }
    }
   },
   "id": "001"
}
```

3.3.2. INIT_RESP

The *INIT_RESP* is a message sent by the Spectrum Database Service (SBS) in response to an initialization request *INIT_RESP* from a White Space Device (WSD). The *INIT_RESP* message confirms whether the initialization request was successfully processed and provides information about the available spectrum for the device.

The *INIT_RESP* follows the following format:

```
{
    "jsonrpc": "2.0",
    "result": {
    "type": "INIT_RESP",
    "version": "1.0",
    "rulesetInfos": [
        {
            "authority": "br",
            "rulesetIds": ["FccTvBandWhiteSpace-2010"],
            "maxLocationChange": 100,
            "maxPollingSecs": 86400
        }
    ]
    },
    "id": "001"
}
```

The *rulesetInfos* field, according to RFC 7545, contains parameters for the ruleset of a regulatory domain that is communicated using the Initialization (5.6)[5]. In addition to the *authority* and *rulesetIds*, it includes optional response fields: *maxLocationChange*[6] and *maxPollingSecs*[7].

[5] https://www.rfc-editor.org/rfc/rfc7545.html#section-5.6

[6] The maximum location change in meters is REQUIRED for the Initialization Response

[7] The maximum duration, in seconds, between requests for available spectrum is REQUIRED for the Initialization Response

3.3.3. Testing

To use the API, two main formats can be used: i) Use the API URL and send the parameters directly from your system, as part of your integrated code, or ii) use calls via *curl*.

The geolocation database is exposed to the outside world via a Web server. For this testing period, the Web server was hosted in a public address at http://200.160.6.8/api/ and it was implemented using the Nginx[8] tools. For testing witch *curl* we present two examples:

Option 1 From file, where `PATH/TO_FILE.json` is the path to a JavaScript Object Notation (JSON) file

```
curl -v -i --header "Content-Type: application/json" --request POST --
    data @PATH/TO_FILE.json URL_HERE
```

Option 2 Inline, here in `INIT_REQ` is just an example

```
curl -X POST -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"spectrum.paws.getSpectrum","params":{"
      type":"INIT_REQ","version":"1.0","deviceDesc":{"serialNumber":"4fbf-
      bae3-c8c5706745e3","fccId":"94d8610a-9eea","rulesetIds":["
      FccTvBandWhiteSpace-2010"]},"location":{"point":{"center":{"latitude
      ":-3.7933105,"longitude":-38.6020162}}},"id":"001"}' \
  URL_HERE
```

Considerasions Some considerations are important for handling this message:

- If the rulesetIds is not sent as an array, the API will return the rule it follows: *FccTvBandWhiteSpace-2010*
- The location used in this message is that of the master
- the *id* field present in messages like *INIT_REQ* or *AVAIL_SPECTRUM_REQ* is assigned by the system that is sending the message, i.e., the White Space Device (WSD), whether it is a master or a slave. This field is used to link a request to its corresponding response.

Params Contains the parameters for the request. For the *INIT_REQ*, it includes the following subfields:

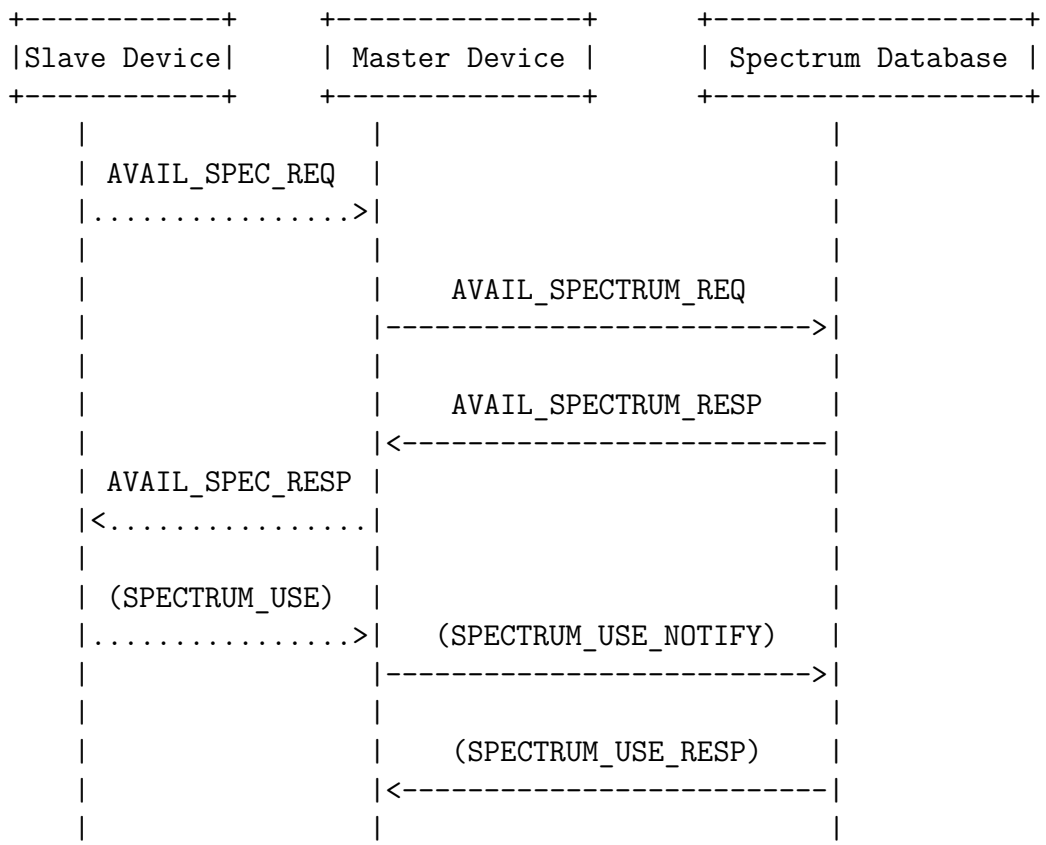
- *serialNumber*: The serial number of the device, used to uniquely identify it.
- *fccId*: The identifier of the device registered with the FCC[9].
- *rulesetIds*: Defines the applicable rulesets for the device, here *FccTvBandWhiteSpace-2010*, indicating it is operating under the FCC 2010 rules for White Space.
- *id*: A unique identifier for the request, used to correlate the response.

[8] https://nginx.org/en/.
[9] Federal Communications Commission

3.4 Available Spectrum Query

The *AVAIL_SPECTRUM_REQ* message is a request sent by a White Space Device (WSD) to a Spectrum Database Service (SBS) to query the availability of TV White Space (TVWS) spectrum in the device's current location. This section outlines our implementation of the Available Spectrum Query messages, as specified in RFC 7545. Figure 5 present the flow of messages. Our implementation currently supports the:

- *AVAIL_SPECTRUM_REQ*,
- *AVAIL_SPECTRUM_RESP*,
- *SPECTRUM_USE_NOTIFY* (simplified) messages.



Figures 5: Sequence of messages exchanged in Avail Spectrum Query

Table 1 show the messages implemented in our solution.

3.4.1. AVAIL_SPECTRUM_REQ

The *AVAIL_SPECTRUM_REQ* follows the following format:

```
1 {
2   "jsonrpc":"2.0",
```

Tables 1: Available Spectrum Query Implementation Status

Message	Implementation
AVAIL_SPECTRUM_REQ	Ok
AVAIL_SPECTRUM_RESP	Ok
AVAIL_SPECTRUM_BATCH_REQ	
AVAIL_SPECTRUM_BATCH_RESP	


```
   "method":"spectrum.paws.getSpectrum",
   "params":{
      "type":"AVAIL_SPECTRUM_REQ",
      "version":"1.0",
      "deviceDesc":{
         "serialNumber":"4fbf-bae3-c8c5706745e3",
         "fccId":"94d8610a-9eea",
         "rulesetIds":["FccTvBandWhiteSpace-2010"]
      },
      "location":{
         "point":{
            "center":{
               "latitude":-3.870961,
               "longitude":-38.664911
            }
         }
      },
      "antenna":{
         "height":76,
         "heightType":"AGL",
         "antennaDirection": {
            "zenith": 10.1,
             "azimuth": 12.1
         },
         "antennaRadiationPattern": [11.1, 22.2],
         "antennaPolarization": "v"
      },
      "masterDeviceLocation":{
         "point":{
            "center":{
               "latitude":-3.7933105,
               "longitude":-38.6020162
            }
         }
      }
   },
   "id":"2"
}
```

The types of variables are:

Field	Data Type	Unit/Format
"jsonrpc"	String	Fixed value: "2.0"
"method"	String	"spectrum.paws.getSpectrum"
"params"	Object	N/A
"type"	String	.ᴬVAIL_SPECTRUM_REQ"
"version"	String	"1.0"
"deviceDesc"	Object	N/A
"serialNumber"	String	Alphanumeric ID
"fccId"	String	Alphanumeric (e.g., FCC ID)
"rulesetIds"	Array of Strings	N/A
"location"	Object	N/A
"point"	Object	N/A
"center"	Object	N/A
"latitude"	Number (Float)	Degrees (-90 to 90)
"longitude"	Number (Float)	Degrees (-180 to 180)
"antenna"	Object	N/A
"height"	Number (Integer)	Meters
"heightType"	String	.ᴬGL.ᵒr .ᴬSL"
"antennaDirection"	Object	N/A
"zenith"	Number (Float)	Degrees (0 to 180)
"azimuth"	Number (Float)	Degrees (0 to 360)
"antennaRadiationPattern"	Array of Numbers (Float)	N/A
"antennaPolarization"	String	"v"(vertical) or "h"(horizontal)
"masterDeviceLocation"	Object	N/A
"point"	Object	N/A
"center"	Object	N/A
"latitude"	Number (Float)	Degrees (-90 to 90)
"longitude"	Number (Float)	Degrees (-180 to 180)
"id"	String	Unique Identifier

Tables 2: Explanation of the Fields of message

3.4.2. AVAIL_SPECTRUM_RESP

```
{
"jsonrpc": "2.0",
"result": {
    "type": "AVAIL_SPECTRUM_RESP",
    "version": "1.0",
    "timestamp": "2024-10-25T09:07:16Z",
    "deviceDesc": {
        "serialNumber": "4fbf-bae3-c8c5706745e3",
```

```
        "fccId": "94d8610a-9eea",
        "rulesetIds": [
            [
                "FccTvBandWhiteSpace-2010"
            ]
        ]
    },
    "spectrumSpecs": [
        {
            "rulesetInfo": {
                "authority": "ANATEL-BR",
                "rulesetId": "FccTvBandWhiteSpace-2010"
            },
            "needsSpectrumReport": false,
            "spectrumSchedules": [
                {
                    "eventTime": {
                        "startTime": "2024-10-25T09:07:16Z",
                        "stopTime": "2024-10-26T09:07:16Z"
                    },
                    "spectra": [
                        {
                            "resolutionBwHz": 6000000,
                            "profiles": [
                                [
                                    {
                                        "hz": 54000000,
                                        "dbm": 14.375
                                    },
                                    {
                                        "hz": 452000000,
                                        "dbm": 25.214844
                                    }
                                ]
                            ]
                        }
                    ]
                }
            ]
        }
    ]
},
"id": "2"
}
```


Testing Option 1 (from file, where `PATH/TO_FILE.json` is the path to a JSON file)

```
curl -v -i --header "Content-Type: application/json" --request POST --
    data @PATH/TO_FILE.json URL_HERE
```

Option 2 (inline, here in `AVAIL_SPECTRUM_REQ` is just an example)

```
curl -X POST URL_HERE \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0","method":"spectrum.paws.getSpectrum","params":{"
      type":"AVAIL_SPECTRUM_REQ","version":"1.0","deviceDesc":{"
      serialNumber":"4fbf-bae3-c8c5706745e3","fccId":"94d8610a-9eea","
      manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3","modelId":"xpto
      -123-123-123","rulesetIds":["FccTvBandWhiteSpace-2010"]},"
      masterDeviceDesc":{"serialNumber":"4fbf-bae3-c8c5706745e3","fccId
      ":"94d8610a-9eea","manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3
      ","modelId":"xpto-123-123-123","rulesetIds":["FccTvBandWhiteSpace
      -2010"]},"location":{"point":{"center":{"latitude":-3.7653855,"
      longitude":-38.5261224}}}},"id":"10"}'
```

3.4.3. SPECTRUM_USE_NOTIFY

SPECTRUM_USE_NOTIFY message is used to notify the Spectrum Database Service (SBS) about the start or end of a White Space Device's (WSD) operation on a TV White Space (TVWS) spectrum channel.

Purpose of *SPECTRUM_USE_NOTIFY*

- The *SPECTRUM_USE_NOTIFY* message is sent by a WSD to inform the SBS that it has begun or stopped using the spectrum at a specific location and frequency range.
- The primary purpose is to keep the spectrum database updated about device operations, enabling the SBS to monitor spectrum usage across various geographical areas and frequency bands.

Usage Context After receiving a positive response to a spectrum request, such as an *AVAIL_SPECTRUM_RESP*, the WSD may start using the allocated TVWS spectrum. Upon starting this operation, the device sends a *SPECTRUM_USE_NOTIFY* to the SBS, indicating it has begun using the spectrum

Key fields and their descriptions

- *params*: This section contains all the parameters required for the notification, in this case: *SPECTRUM_USE_NOTIFY*
- *location*: Provides the geographical location of the notifying device.
- *spectra*: Details the frequency spectrum being used by the device: i) *resolutionBwHz* is the resolution bandwidth in Hz and ii) *profiles*, contains a list of frequency power profile (in hz and dbm). The Table 3 presents the data types and the units used in spectra.

The *SPECTRUM_USE_NOTIFY* follows the following format:

```
{
   "jsonrpc":"2.0",
   "method":"spectrum.paws.notifySpectrumUse",
   "params":{
      "type":"SPECTRUM_USE_NOTIFY",
      "version":"1.0",
      "deviceDesc":{
         "serialNumber":"4fbf-bae3-c8c5706745e3",
         "fccId":"94d8610a-9eea",
         "rulesetIds":[
            "FccTvBandWhiteSpace-2010"
         ]
      },
      "location":{
         "point":{
            "center":{
               "latitude":-3.7933105,
               "longitude":-38.6020162
            }
         }
      },
      "masterDeviceDesc":{
         "serialNumber":"4fbf-bae3-c8c5706745e3",
         "fccId":"94d8610a-9eea",
         "rulesetIds":[
            "FccTvBandWhiteSpace-2010"
         ]
      },
      "masterDeviceLocation":{
         "point":{
            "center":{
               "latitude":-3.7933105,
               "longitude":-38.6020162
            }
         }
      },
      "spectra":[
         {
            "resolutionBwHz":6000000,
            "profiles":[
               [
                  {
                     "hz":54000000,
                     "dbm":29
                  },
                  {
                     "hz":60000000,
```

```
                    "dbm":29
                }
            ]
        ]
    }
  ]
},
"id":"002"
}
```


Field	Data Type	Unit/Format
`resolutionBwHz`	Integer/Float	Hertz (Hz)
`profiles`	Array of Arrays of Objects	List of frequency-power profiles
`hz` (within `profiles`)	Integer/Float	Hertz (Hz)
`dbm` (within `profiles`)	Integer/Float	Decibel-milliwatts (dBm)

Tables 3: Explanation of the `spectra` Fields

4 Customizations

Some customizations were implemented in the TVWS support provided by the API, notably in the areas of antennas, session management, and authentication.

4.1 Antenna

This section introduces a set of new attributes designed to enhance communication capabilities. Specifically, the *AntennaCharacteristics*[10] parameter of the PAWS Protocol to implement an optimized usage of the spectrum.

4.1.1. AntennaCharacteristics

The basic AntennaCharacteristics of RFC 7545 covers essential parameters detailed in the schema below.

```
+-----------------------------------+
|AntennaCharacteristics             |
+-------------------------+---------+
|height:float             | OPTIONAL |
|heightType:enum          | OPTIONAL |
|heightUncertainty:float  | OPTIONAL |
|.........................|.........|
|*characteristics:        | OPTIONAL |
|   various               |          |
```

[10] https://www.rfc-editor.org/rfc/rfc7545.html#section-5.3

```
+------------------------+----------+
```

The default schema (presented above) includes the following parameters:

- Height (height): This parameter represents the physical height of the antenna above the ground or a reference point. It is typically measured in **meters**. The height significantly influences the antenna's coverage area and interaction with its surroundings.
- Height Type (heightType): An optional field that specifies the type of height measurement. Common types include:
  - Above Ground Level (AGL)
  - Above Mean Sea Level (AMSL)
- Height Uncertainty (heightUncertainty): Another optional field that quantifies the uncertainty associated with the height measurement. It accounts for inaccuracies or variations in the height value.

The literature reports that when designing and deploying wireless communication systems, accurate modeling of antenna behavior is essential for efficient spectrum utilization and user protection. While basic parameters like antenna height provide a foundation, more is needed to capture the intricacies of real-world scenarios.

To create a more precise model[11], therefore promoting better user protection (primary and secondary) and an efficacy spectrum usage, we incorporate the following optional[12] fields to enhance the system performance:

- antenna direction
- antenna radiation pattern
- antenna gain (extracted from the radiation pattern)
- antenna polarization

The code fragment below ilustrates our implementation of the optional fields.

```
{
   "jsonrpc":"2.0",
   "method":"spectrum.paws.getSpectrum",
   "params":{
      "type":"AVAIL_SPECTRUM_REQ",
      "version":"1.0",
        (...)
      "antenna":{
         "height":76,
```

[11] In terms of propagation: coverage and interference protection

[12] These optional fields (antenna direction, radiation pattern, gain and polarization) are presented in section 5.3 of the RFC7545

```
        "heightType":"AGL",
        "antennaDirection": {
           "zenith": 10.1,
           "azimuth": 12.1
        },
        "antennaRadiationPattern": [11.1, ..., 22.2],
        "antennaPolarization": "v"
      },
      (...)
    },
    "id": "001"
}
```

We use the Azimuth and Zenith to represent antenna direction. The Azimuth is an angle (double/float) determined using the geographic north (0,0).

The $AntennaRadiantionPatern$ is represented by considering gains at specific azimuth angles every 5 degreea to create a complete radiation pattern. This specification results in 72 double/float position lists, indicating a gain of 5 degrees.

The Antenna Gain will be expressed in **dBi**.

For example, Gain_AZ[0] indicates the gain from the Azimuth in 0 degrees to 5 degrees. The Gain_AZ[72] indicates gain from Azimuth in 355°to 360°.

Default Values

- if the user does not send the AntennaCharacteristics the system will consider an **isotropic antena with zero gain**.
- If the user does not send the complete **72 positions** of the radion pattern gain, thus an isotropic antenna is assumed with gain equal to the maximum value within the array.

4.1.2. antennaPolarization

We also represent the polarization, witch affect the electromagnetic waves propagation. Using the following flags (See datatype in Table 2):

- *v* : Vertical
- *h* : Horizontal
- *e*: Elliptical
- *v,h*: Vertical and Horizontal, and it will consists an array of arrays

If the value sent in the message does not correspond to one of these two values (string and lowercase), the system will return an error message, indicating that something is wrong, as shown in the following image. If no polarization parameter is entered our system will consider the **horizontal polarazition**, *h*.

4.1.3. antennaRadiationPattern

The `antennaRadiationPattern` uses the antenna polarization as an index. If the polarization is specified as `"v"`, `"h"`, or `"e"`, the array in `antennaRadiationPattern` represents the gain per radial (as described in a previous section) corresponding to that polarization.

However, if the polarization is specified as `"v,h"`, then `antennaRadiationPattern` is defined as an array of arrays:

- The **outer array** contains two elements, one for each polarization;
- Each **inner array** contains 72 values, corresponding to the gain per radial direction.

Each value in `antennaRadiationPattern` is calculated as:

$$-10\log_{10}\left(\left(\frac{E_x}{E_{\text{max}}}\right)^2\right)$$

where $x = h$ or $x = v$, representing the horizontal or vertical components of the electric field, respectively.

In the case of elliptical polarization, it is assumed that the combination of the horizontal (E_h) and vertical (E_v) electric field components for each radial direction has already been computed. The corresponding value is given by:

$$-10\log_{10}\left(\left(\frac{E_e}{E_{\text{max}}}\right)^2\right) = -10\log_{10}\left(\left(\frac{E_h}{E_{\text{max}}}\right)^2 \times \left(\frac{E_v}{E_{\text{max}}}\right)^2\right)$$

4.2 Sessions

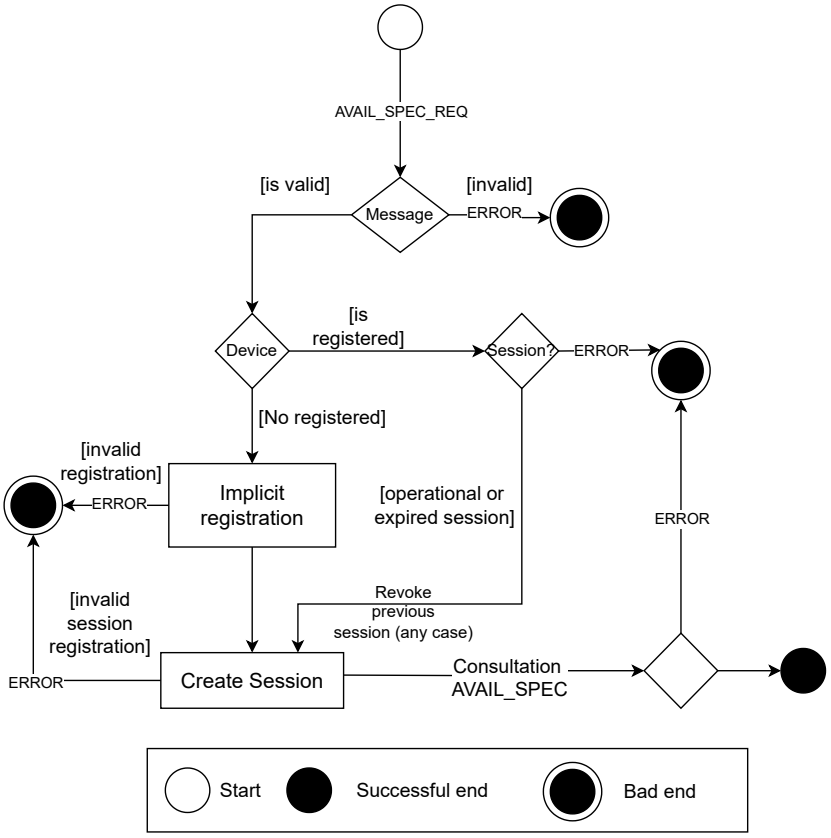
In the context of APIs based on the PAWS (Protocol to Access White Space) standard, as defined in RFC 7545, the use of sessions plays a crucial role in efficiently and securely managing spectrum access by authorized devices.

Sessions allow for maintaining a persistent state between the spectrum database and the devices during a communication period. This is essential to ensure that decisions regarding spectrum usage are made based on the device's interaction history and context.

In our implementation, each device is identified by a unique set of attributes: i) *serinalNumber*, ii) *manufacturerId* and iii) *modelId*. These identifiers are used to link a specific session to a physical device, enabling the API to accurately recognize which device is making the request, even across multiple connections over time. This provides several key benefits:

- State Management: Allows tracking of request and response history, helping maintain compliance with spectrum usage regulations.
- Security and Authenticity: Reduces the risk of fraudulent or malicious requests, since each session is tied to a validated device.
- Response Efficiency: Enables optimizations such as response caching or applying differentiated usage policies per session.
- Auditing and Monitoring: Facilitates analysis of spectrum usage patterns, detection of anomalies, and operational troubleshooting.

See Figure 6 for an example of the implementation of implicit session creation.



Figures 6: Implicit Session Creation. "Note that upon receiving the message, the API checks whether a registration already exists for that device. If no registration is found, the parameters provided in the message are used to create the device registration (implicit registration), followed by the session creation and the corresponding response to the message

The session implementation follows two possible flows: i) When a Registration_REQ is sent to the database, this is referred to as explicit session creation, or ii) When an Avail_Spectrum_Req is sent, in which case the session is created implicitly.

Since *Registration_Req* is optional, but *Avail_Spectrum_Req* is mandatory, we will present a diagram below that explains how the session is created via *Avail_Spectrum_Req* using the concept of an implicit *Registration_Req*.

4.3 Authentication

This subsection present the proposed model for authentication, registration, and *token* management within the TV White Space (TVWS) API ecosystem, inspired by *2.1.7 Client Auth Candidate 2: Shared Secret* from the document *Brazil TVWS – Phase 2, version dated September 17, 2025*. In this context, UUID tokens will be used as the mechanism for authentication. Tokens will be assigned to both Operators and Devices, in accordance with section 2.1.7.

4.3.1. Terminology and Core Concepts

Table 4 introduces the main terms used in the TVWS architecture and their meanings. Furthermore, the table presents some blockchain-specific terms, we will not delve into these terms or the operational mechanisms underlying this technology.

Tables 4: Main terminology used in the TVWS architecture

Term	Description
Checkin	Registration of the operator to receive the auth token
wallet	Blockchain account (DBTVWS) associated with an operator.
address	Unique identifier for an account on the blockchain.

4.3.2. Token Assigment for Operators and Devices

The allocation of tokens to operators and devices involves an additional level of context, as operators are issued a credential, called a wallet, which allows them to assign tokens to devices. This section briefly presents information about the Token Format, the Operator checkin Flow, and how authentication will be handled during PAWS messages, in Phase II.

Token Format

The authentication model uses UUID[13] v4 tokens, prefixed with metadata identifying the operator and token type.

```
Example:
3f4e9c80-2c47-4a9d-b8d2-f0c5bb7d5aaf
8c02c5d0-9e81-47d0-8617-7df49ce8efb3
```

Internally, this token is stored in a database table along with its metadata. In the Token Database Schema, we present the fields of the table that stores the data for authentication.

4.3.3. Token Database Schema

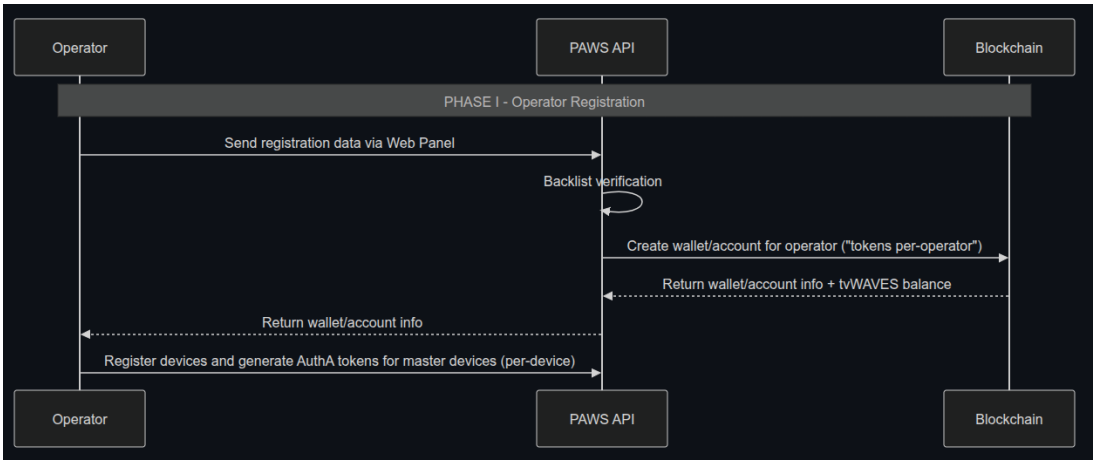
Field	Type	Description
token	string	Full token
scope	enum(operator,device)	token scope: `O` (operator) or `D` (device)
operator_id	string	vat_number without non-numeric characters
device_id	string (nullable)	Device identifier (serialNumber)
model_id	string (nullable)	modelID in devideDesc
manufacturer_id	string	ManufacturerId in deviceDesc
created_at	date	Issue date
expires_at	date	Expiration date
revoked	boolean	Revocation flag

[13] **UUID:** random 128-bit identifier

contact	string(nullable/device)	email admin who issued the token
wallet	string (nullable)	Account in blockchain
public key	string (nullable)	Public wallet key
private key	string (nullable)	Private wallete key
balance	string	balance governance token
description	string (nullable)	Optional management description
version	string	authentication version control information

4.3.4. Operator Checkin Flow

The following diagram illustrates the operator registration and wallet creation process.



Figures 7: Operator Registration Flow

The Figure 7 illustrates the three entities involved in the operator check-in and token assignment process. Initially, the operator, via JSON and API, submits their registration data to create an account to access the database resources. The API processes this check-in message and verifies whether the CNPJ (Brazilian National Registry of Legal Entities) confirms that the company identifier is correct, valid, and authorized to operate. If the verification is successful, the API sends the data for the creation of a wallet linked to the CNPJ. The API then stores this information and returns the operator's authorization token.

Below, we present an example of the JSON sent for the operator check-in.

Note that the *CNPJ* sent in the JSON will correspond to the *operator_id* identified in the table.

Listing 1: Example JSON Operator checkin request

```
{
   "vat_number": "123.345.678/999-11",
   "email": "company@companya.com.br",
   "name": "Company A",
   "version" : "basic2025a"
}
```

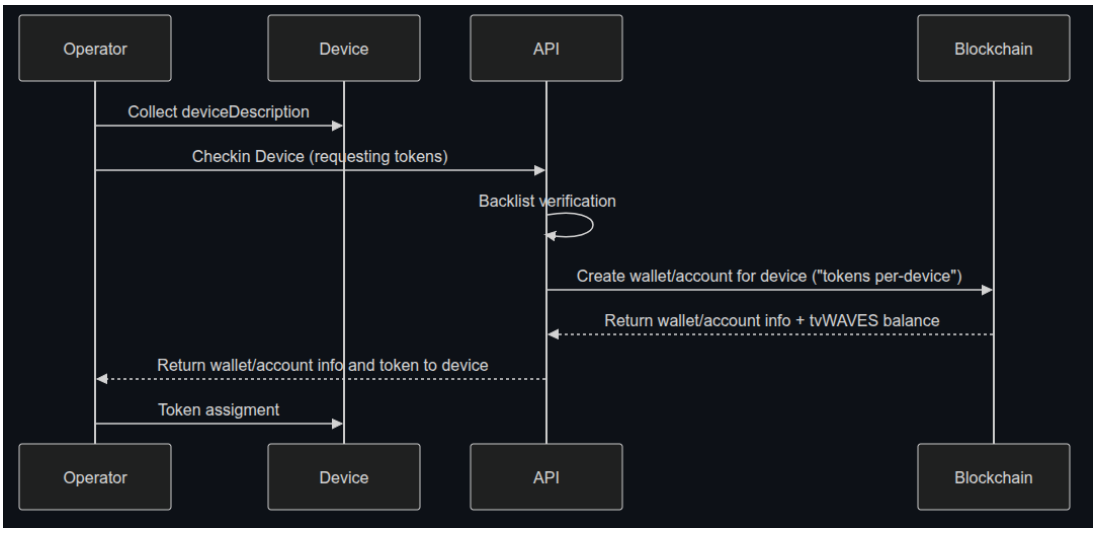
Response:

Listing 2: Example JSON Operator checkin response

```
{
   "success": true,
   "message": "Operator successfully authorized.",
   "token": "22bd803f-a953-489d-a311-e7e40f5d87a9",
   "id": 6
}
```

4.3.5. Device Token Assigment

The allocation of tokens to devices is requested by the operator and is currently performed on a per-device basis. The flow illustrated in the figure 8 shows the process of requesting and assigning tokens for each device.



Figures 8: Flow for requesting and assigning authentication tokens to consume the TVWS API

The operator collects information from the device, such as those found in *deviceDescription*[14]. This information is sent to the API, where a verification of the device data is performed. If the verification is successful, the API requests the creation of a wallet for the device on the blockchain. Finally, the account information and device tokens are sent to the operator, who then assigns the token to the device.

Below, we present an example of the JSON sent by the operator for the device checkin.

Listing 3: Example JSON Device checkin

```
{
   "operator_id":"123.345.678/999-11",
   "token": "22bd803f-a953-489d-a311-e7e40f5d87a9"
```

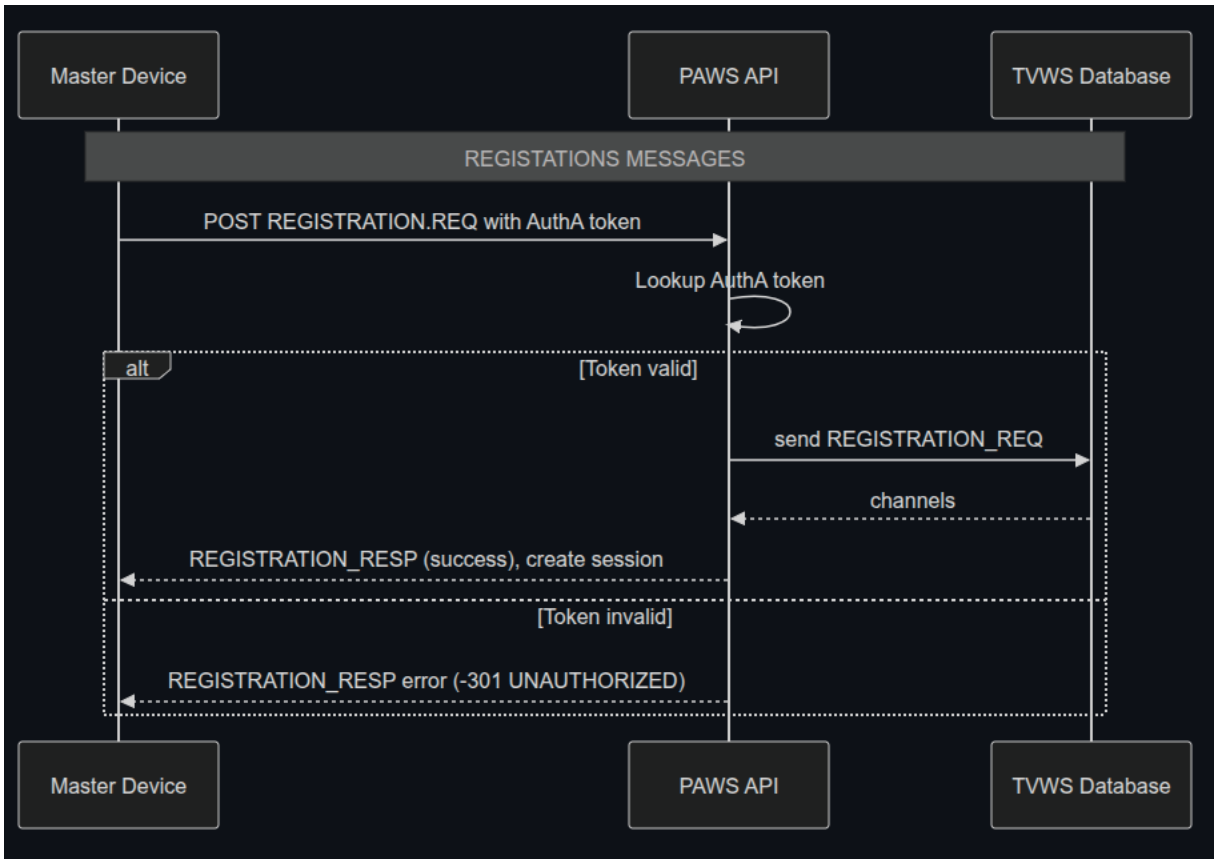
[14] https://www.rfc-editor.org/rfc/rfc7545.html#page-37

```
   "deviceDesc":{
      "serialNumber":"4fbf-bae3-c8c5706745e3",
      "fccId":"94d8610a-9eea",
      "manufacturerId":"123-ABCD-4fbf-bae3-c8c5706745e3",
      "modelId":"xpto-123-123-123",
      "rulesetIds":[
         "FccTvBandWhiteSpace-2010"
      ]
   },
   "version":"basic2025a"
}
```

4.3.6. Authentication and PAWS messages

In this session, we will present the authentication process during PAWS message exchanges.
REGISTRATION_REQ

During registration, the master device sends its registration request using the `AuthA` token. The API validates the token, performs lookups, and creates a session for the device.



Figures 9: PAWS: Registration Request Flow

4.3.7. Practical Example

We will now address apractical examples of using the token for authentication.

Equivalent Curl Command

General example:

```
curl -X POST https://urlAPI/paws \
  -H "Authorization: Bearer 8c02c5d0-9e81-47d0-8617-7df49ce8efb3" \
  -H "Content-Type: application/json" \
  -d '{ ... }'
```

Full example

```
curl -X POST URL_HERE \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer 8c02c5d0-9e81-47d0-8617-7df49ce8efb3" \
  -d '{"jsonrpc":"2.0","method":"spectrum.paws.getSpectrum","params":{"type":"
     REGISTRATION_REQ","version":"1.0","deviceDesc":{"serialNumber":"b4fbf-
     bae3-c8c5706745e3","fccId":"94d8610a-9eea","manufacturerId":"123-ABCD-4
     fbf-bae3-c8c5706745e3","modelId":"xpto-123-123-123","rulesetIds":["
     FccTvBandWhiteSpace-2010"]},"location":{"point":{"center":{"latitude
     ":-3.782542,"longitude":-38.556012}}}},"id":"10"}'
```

4.3.8. AVAIL_SPECTRUM_REQ

The following flow describes how master and slave devices interact when querying for available spectrum channels. Token validation ensures that all devices belong to the same operator wallet.

5 Administrative Services

Some administrative services are provided through endpoints that allow other capabilities of the API to be utilized. The following are some examples.

GET /admin/test

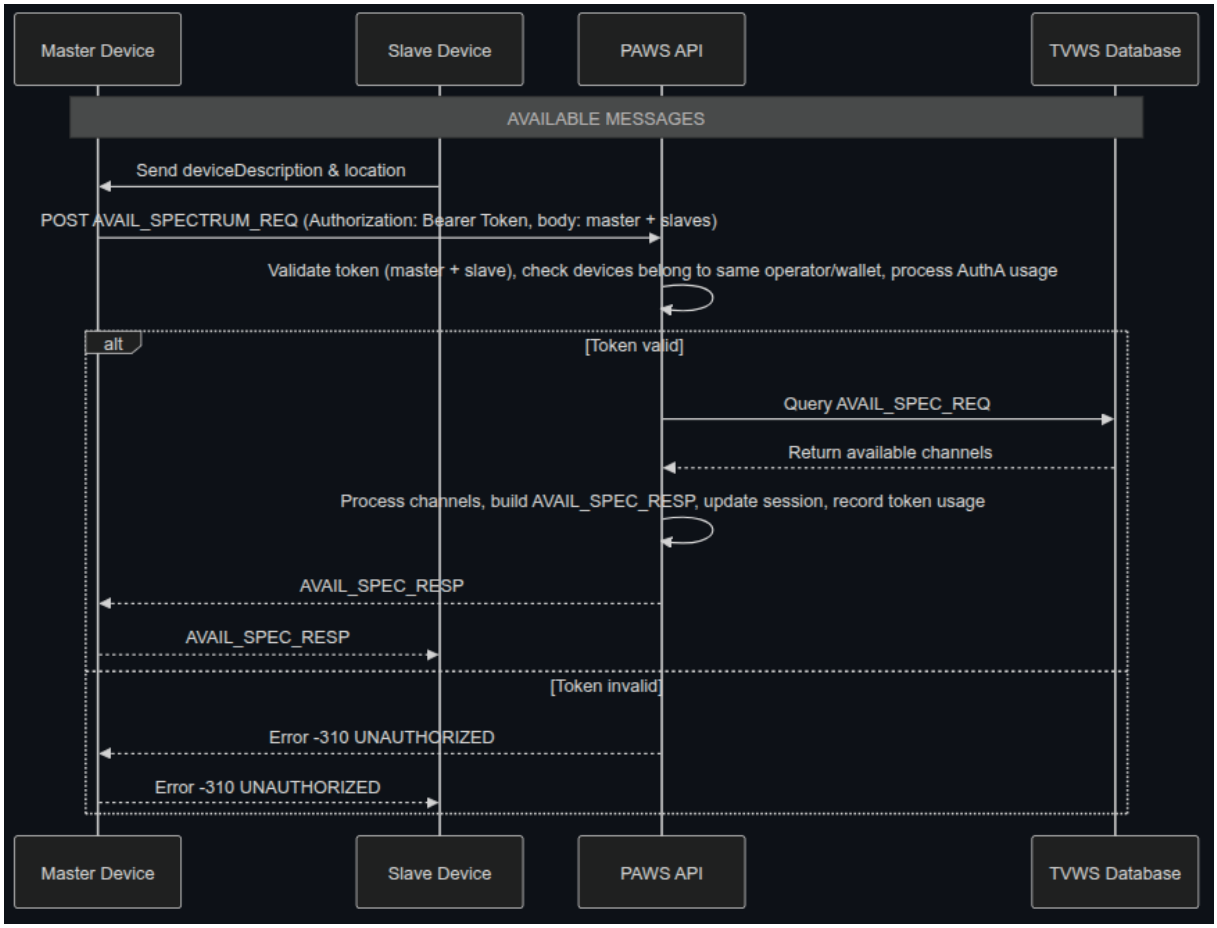
Description: Health check to verify that the server is running.
Response:

```
1 "up"
```

GET /admin/get?location_id=pixel

Query Parameters:

- `location_id` (string)



Figures 10: PAWS: Registration Request Flow

Description: Retrieve blockchain location data and available channels.
Example: /admin/get?location_id=71fce8a17ac51196b06d57187f2c7ad9

Response:

```
{
  "loc_id": "cf73bc...",
  "location_data": {
    "id": "statedId",
    "cityId": "cityId",
    "topLeft": [latitude, longitude],
    "bottomRight": [latitude, longitude],
    "sinr": [],
    "tvNoiseFloor": [],
    "protectedChannels": [],
    "tvTransmitters": []
  },
  "available_channels": [... ]
```

```
}
```

GET /admin/get-location/:lat/:lon

Path Parameters:

- `lat` (float)
- `lon` (float)

Description: Find location ID by latitude and longitude.
Response:

```
{"pixel": "location_id" }
```

GET /admin/location/all

Description: Get a list of all registered location IDs.
Response:

```
{
  "location_list": {
    "id1": "up",
    "id2": "up"
  }
}
```

GET /admin/location/all/detail

Description: Get detailed information for all locations.
Response:

```
{
 "location_list": {
    "784962210b4f31c33154f38510400866": {
        "cityId": 149,
        "top_left": {
            "latitude": -5.0295,
            "longitude": -38.7975
        },
        "bottom_right": {
            "latitude": -5.04,
            "longitude": -38.808
        },
        "sinr": [0,0,0,-2.91, 3.61],
        "protected_channels": [false, false]
```

```
    }
  }
}
```

GET /admin/lock

Query Parameters:

- `location_id` (string)
- `channel` (int)

Description: Lock a channel in a specific location on-chain.
Response:

```
{
  "location_id": "cf73bc...",
  "status": "200",
  "transaction_id": "...",
  "transaction_url": "https://explorer.solana.com/tx/..."
}
```

GET /admin/transmitter/all

Description: List all transmitters and their properties.
Response:

```
{
  "mosaico_id": {
    "pda": "...",
    "status": "...",
    "class": "...",
    "channel": "...",
    "position": "...",
    "height": "...",
    "hz": "...",
    "erp": "..."
  }
}
```

POST /admin/path

Description: Compute all pixels along a straight line between two points and return elevation.
Request Body:

```
{
  "point_a": {"longitude": -xx, "latitude": yy },
  "point_b": {"longitude": -xx, "latitude": yy }
}
```

Response:

```
[
  {
    "pixel_id": "pixel_1",
    "latitude": xxx,
    "longitude": yyy,
    "height": 57.12
  },
  ...
]
```

POST /admin/hrpath

Description: Calculate terrain elevation between two points using higher resolution.
Request Body:

```
{
  "point_a": {"longitude": -xx, "latitude": yy },
  "point_b": {"longitude": -xx, "latitude": yy }
}
```

Response:

```
[
  {
    "pixel_id": "-",
    "latitude": yy,
    "longitude": -xx,
    "height": 45.32
  },
  ...
]
```

GET /admin/session

Description: Retrieve all session records stored in the database.
Response:

```
[
  {
    "sid": "1741770350716",
```

```
    "sess": {"route": "/paws" },
    "created_at": "2025-03-12T09:05:50.714Z",
    "expire": "2025-03-12T09:07:50.716Z",
    "device": {
        "serialNumber": "df7016ed-bc8d-4884-aace-e621de8a98e2",
        "fccId": "ABCYXYNO00C6O",
        "manufacturerId": "123-ABCD-4fbf-bae3-c8c5706745e3",
        "modelId": "xpto-123-123-123"
    },
    "spectrumspecs": [
        {
            "rulesetInfo": {
                "authority": "ANATEL-BR",
                "rulesetId": [
                    "FccTvBandWhiteSpace-2010"
                ]
            },
            "spectrumSchedules": [
                {
                    "spectra": [
                        {
                            "profiles": [
                                [{"hz": 54000000,"dbm": 14.375} ]
                            ],
                            "resolutionBwHz": 6000000
                        }
                    ],
                    "eventTime": {
                        "stopTime": "2025-03-12T09:07:50Z",
                        "startTime": "2025-03-12T09:05:50Z"
                    }
                }
            ],
            "needsSpectrumReport": true
        }
    ],
    "type": ["AVAIL_SPECTRUM_REQ"]
  }
]
```


GET /admin/list-tables

Description: List all PostgreSQL tables in the public schema.
Response:


```
[
```

```
    {"table_name": "geography_columns" },
    {"table_name": "geometry_columns" },
    {"table_name": "spatial_ref_sys" },
    {"table_name": "session" },
    {"table_name": "telecom_location" },
    {"table_name": "tvws_dbs" }
]
```

6 Deploy

This section discusses two important aspects of the deployment and is not intended as a guide on how to perform the deployment; that information is provided in the code documentation.

6.1 Configuration file

The `default.json` file defines the global configuration parameters of the system. It centralizes operational, infrastructural, and functional settings required for the correct execution of the API and its associated services. Each top-level key is described below.

6.1.1. `blockchain`

This section contains all parameters related to blockchain interaction. It defines authentication credentials and external references required for on-chain operations. The configuration includes the network endpoint used to communicate with the blockchain and the private key employed for signing transactions, as well as the base URL of the blockchain transaction explorer used for monitoring and verification purposes.

```
"blockchain": {
  "auth": {
    "net": "http://ip:8899",
    "privateKey": [... ]
  },
  "urlSolanaExplorerTx": "https://explorer.solana.com/tx/"
}
```

The `blockchain` section defines all parameters required for interaction with the underlying blockchain infrastructure.

- `auth`: Groups authentication-related parameters.
  - `net`: Specifies the blockchain network endpoint used to submit and query transactions.
  - `privateKey`: Represents the private key used to cryptographically sign blockchain transactions. This key must be securely stored and restricted, as it grants full signing authority.

- `urlSolanaExplorerTx`: Defines the base URL of the blockchain explorer used to inspect and audit submitted transactions.

6.1.2. `server`

The `server` section defines API-level operational parameters and external service URLs. It specifies the port on which the API is exposed, monitoring and alerting mechanisms (including email-based notifications), and the base URLs used to access administrative services and propagation-related modules.

```
"server": {
  "apiParameterization": {
    "apiPort": 6000,
    "monitoringApi": {
      "email": {
        "serviceAlert": true,
        "emailUser": "...",
        "emailPassword": "...",
        "emailToAlert": "..."
      }
    }
  },
  "urlParametrization": {
    "urlBaseServer": "...",
    "urlPathAdminService": "...",
    "urlPropagationModuleMaxPower": "..."
  }
}
```

The `server` section controls runtime behavior and external connectivity of the API.

- `apiParameterization`: Defines core API runtime settings.
  - `apiPort`: TCP port on which the API service is exposed.
  - `monitoringApi.email`: Configures email-based alerting.
    - `serviceAlert`: Enables or disables alert notifications.
    - `emailUser` and `emailPassword`: Credentials used by the alerting service.
    - `emailToAlert`: Recipient address for alerts.
- `urlParametrization`: Specifies external service endpoints required by the API, including administrative services and propagation-related modules.

6.1.3. `authentication`

This section controls the authentication and authorization behavior of the system. It defines whether authentication is enabled for operators, devices, and administrative users, as well as the expiration times associated with each authentication context. These parameters allow fine-grained control over access security and session validity.

```
"authentication": {
  "authentication_activation": false,
  "timeExpireAuthOperator": 240,
  "timeExpireAuthDevice": 300,
  "admin_authentication_activation": false,
  "ExpireAuthAdmin": 3600
}
```

This section defines authentication policies and expiration rules.

- `authentication_activation`: Enables or disables authentication for operators and devices.
- `timeExpireAuthOperator`: Authentication validity period for operators, in seconds.
- `timeExpireAuthDevice`: Authentication validity period for devices, in seconds.
- `admin_authentication_activation`: Enables or disables administrative authentication.
- `ExpireAuthAdmin`: Expiration time for administrative authentication sessions.

6.1.4. `database`

The `database` configuration specifies the connection parameters for the geospatial database used by the system. It includes credentials, host address, port, and database name, enabling persistent storage and retrieval of geographic and operational data.

```
"database": {
  "geoDatabase": {
    "user": "postgres",
    "password": "postgres",
    "host": "localhost",
    "port": 5432,
    "database": "tvws"
  }
}
```

The `database` section defines the connection parameters for the geospatial database.

- `geoDatabase`: Contains credentials and network parameters used to connect to the PostgreSQL database responsible for storing geographic and operational data.

6.1.5. `test`

This section contains parameters related to testing and performance evaluation. In particular, it allows the activation of execution time measurement for different API components, supporting diagnostics and performance analysis during development or validation phases.

```
"test": {
  "timeMetrics": {
    "description": "...",
    "recordExecTime": false
  }
}
```

This section supports testing and diagnostics.

- `timeMetrics`: Controls execution time measurement.
- `recordExecTime`: When enabled, records performance metrics for internal API components.

6.1.6. `propagationModule`

The `propagationModule` section defines parameters associated with radio propagation modeling and power calculation. It includes numerical parameters used internally by the propagation algorithms, as well as the configuration of an external propagation service, such as its URL, signal-to-interference thresholds, and maximum transmission power constraints.

```
"propagationModule": {
  "propagationParameters": {
    "numSplits": 10,
    "channelSinMax": 27,
    "distancePoints": 30,
    "maxTxPower": 30
  },
  "propagationService": {
    "url": "...",
    "max_tx_power": 30,
    "sinr_border": 27,
    "query_distance_in_km": 30
  }
}
```

The `propagationModule` section configures radio propagation analysis.

- `propagationParameters`: Defines internal algorithmic parameters such as SINR thresholds, distance resolution, and maximum transmission power.
- `propagationService`: Specifies the external propagation service endpoint and operational limits used during power and coverage calculations.

6.1.7. `antennaParameters`

This section defines configuration limits related to antenna modeling. In particular, it specifies the maximum supported size for antenna radiation pattern arrays, ensuring consistency and preventing oversized data structures during processing.

```
"antennaParameters": {
  "maxSizeArrays": 72
}
```

This section defines constraints related to antenna modeling.

- `maxSizeArrays`: Maximum supported size for antenna radiation pattern arrays.

6.1.8. `standardResponseMessage`

The `standardResponseMessage` section defines default values used in PAWS-compliant responses generated by the API. It includes protocol versioning, ruleset information, spectrum constraints, bandwidth resolution, and API version identification. These parameters ensure standardized and interoperable responses across different PAWS interactions.

```
"standardResponseMessage": {
  "jsonrpc": "2.0",
  "version": "1.0",
  "rulesetInfos": {... },
  "rulesetInfo": {... },
  "needsSpectrumReport": true,
  "maxTotalBwHz": 6000000,
  "spectra": {"resolutionBwHz": 6000000},
  "versionApi": "2025.11a"
}
```

This section defines default PAWS-compliant response parameters, including protocol versions, regulatory rulesets, bandwidth constraints, and API versioning. These values ensure interoperability and regulatory compliance across spectrum access interactions.

6.1.9. `session`

This section controls session management behavior. It defines cryptographic secrets, expiration times for different session types, cookie lifetime, session pruning intervals, and whether session management is enabled. These parameters govern how client sessions are created, maintained, and invalidated.

```
"session": {
  "session_secret": "...",
  "time_expire_session_avail": 120,
  "time_expire_session_init": 180,
  "time_expire_session_registration": 180,
  "time_expire_cookie": 60000,
  "pruneSessionInterval": 60,
  "session_activation": false
}
```

This section governs session lifecycle management.

- Cryptographic secrets and expiration times define how long sessions remain valid.

- `pruneSessionInterval` controls how frequently expired sessions are removed.
- `session_activation` enables or disables session tracking globally.

6.1.10. `csv`

The `csv` section defines formatting parameters used when processing CSV files. It specifies delimiters for fields, lists, and lines, ensuring consistent parsing and generation of CSV-based data exchanges.

```
"csv": {
  "delimiter": ",",
  "list_sparator": ";",
  "line_separator": "\n"
}
```

The `csv` section defines formatting rules for CSV file parsing and generation, ensuring consistent handling of structured text-based data.

6.2 Testing

In order to ensure that the API is functioning correctly, a set of automated tests was developed, comprising several dozen test cases that evaluate the state of the API, covering both support for PAWS messages and management services exposed through the administrative routes. Instructions on how to execute these tests are documented in the project repository.